THE UNIVERSITY OF BRITISH COLUMBIA
Electrical and Computer Engineering Department

CPEN 442 **Introduction to Cybersecurity** **Winter Term 1, 2026**

Assignment 3: Authentication and Hashing

Name	Student number
Chongkai (Lucas) Lyu	30349559
Leyang Gao	58256207

Questions 3, 6, and 9 are answered in `a3.py`; this PDF covers Questions 1, 2, 4, 5, 7, and 8.

1 Offline Dictionary Attacks

[10 pts] **Question 1** Which hash function generated each of the three digests, how did you deduce it, and what are the passwords?

Digest 1: hash function `SHA-256`, password `annette`
Digest 2: hash function `SHA3-224`, password `spurs`
Digest 3: hash function `SHA-1`, password `loki`
How I deduced the hash functions: One hexadecimal character is 4 bits, so the bit length of a digest is four times its character length. The three digests have lengths 64, 56, and 40, which are 256, 224, and 160 bits. That narrows the SHA family, but it does not name a single function: SHA-256, SHA3-256, and SHA-512/256 all produce 64 hex characters, and SHA-224, SHA3-224, and SHA-512/224 all produce 56. Only SHA-1 is a common SHA function with a 40-character digest.
The passwords cannot be recovered by inverting the hash. The assignment says every password comes from the dictionary, so I hashed each dictionary word with every candidate of the matching length and compared the result with the digest. Each digest matched exactly once: SHA-256(`annette`), SHA3-224(`spurs`), and SHA-1(`loki`). The same-length alternatives matched no dictionary word, so the dictionary search both identifies the algorithm and recovers the password.

[10 pts] **Question 2** Answer the three questions about the bcrypt digests, briefly explaining each answer (one sentence is enough).

Hash function used: bcrypt. All three digests use bcrypt, as indicated by the `$2b$` prefix in their Modular Crypt Format representations.
Same salt or different salts: different salts. The 22-character salt fields following the cost factors are `AS2K2H8nSwdvs2yNoDgtbu`, `Shq4mPuANPJbdqozQkB1NO`, and `dHO22WXdr4/w6PiDHhVYA.`
Digest that took longer to compute, and why: digest 2. Its cost factor is 8, while digests 1 and 3 have cost factor 6. bcrypt's work scales as 2^{cost}, so digest 2 takes about $2^8/2^6 = 4$ times as much work.

[15 pts] Question 4 Which of `q3_sample1.txt`, `q3_sample2.txt`, and `q3_sample3.txt` is the easiest to break, and which is the hardest? Why?

Easiest: `q3_sample1.txt`
Explanation: All three files contain 10 bcrypt digests with version `2b` and cost factor 4, so each hash call has essentially the same cost. `q3_sample1.txt` uses one salt for all 10 digests: the first 29 characters are identical. Hashing a candidate once with that salt and cost lets me compare the result with every digest. On a full scan of a 1000-word dictionary, that is 1×1000 bcrypt calls.

Hardest: `q3_sample2.txt`
Explanation: `q3_sample2.txt` uses 10 different salts, so the same candidate must be hashed separately for each digest and the computation cannot be reused. A full scan of the same 1000-word dictionary takes 10×1000 calls. `q3_sample3.txt` is in between: five salts, each shared by two digests, so a full scan takes about 5×1000 calls. A single run can spend more time on sample 3 if those passwords occur later in the dictionary and the search stops when they are found. That changes how far down the dictionary the attack walks, not the fact that sample 2 has no salt to share.

2 Offline Brute-Force Attacks

[10 pts] Question 5 What is the maximum password length that your computer can brute-force in under one day, for bcrypt (4 rounds) and for SHA-256? Explain how you got each number.

My computer's hashing speed: bcrypt (4 rounds): 1065.9 hashes per second; SHA-256: 765992.0 hashes per second
Maximum length with bcrypt: 4
Maximum length with SHA-256: 5
How I got these numbers: I ran `q5_test.py` with `hash_number = 10000`. The alphabet is `string.digits + string.ascii_letters + string.punctuation`, which

has 94 characters. One day is 86400 seconds, so this computer can compute $1065.9 \times 86400 \approx 9.21 \times 10^7$ bcrypt hashes, or $765992 \times 86400 \approx 6.62 \times 10^{10}$ SHA-256 hashes, in under one day. The number of passwords of length L is 94^L.
For bcrypt, $94^4 = 78074896$ hashes take $78074896/1065.9 \approx 73248$ seconds, about 0.85 days. 94^5 takes about 79.7 days, so 4 is the longest length that finishes in under one day.
For SHA-256, $94^5 = 7339040224$ hashes take $7339040224/765992 \approx 9581$ seconds, about 0.11 days. 94^6 takes about 10.4 days, so 5 is the longest length.

3 Two-Factor Authentication

[10 pts] Question 7 Which service did you pick, which forms of two-factor authentication does it support, and how could an attacker circumvent it?

Service: Amazon Web Services (AWS account sign-in).
Forms of two-factor authentication it supports: AWS IAM supports three MFA types, which matches the Software and Hardware columns on 2fa.directory. AWS IAM does not offer SMS or email codes as standard MFA enrollment methods.

- Virtual authenticator apps that generate a six-digit TOTP code, such as Google Authenticator, Microsoft Authenticator, or Authy.
- Hardware TOTP tokens purchased for use with AWS.
- Passkeys and FIDO2 security keys, including a hardware key such as a YubiKey and a platform authenticator unlocked with a fingerprint, face, or device PIN.

How an attacker could circumvent it (a realistic example): Suppose the second factor is a virtual authenticator app. An attacker sends a phishing email about an AWS billing alert. The link opens a reverse-proxy page that looks like the AWS console and forwards each entry to the real sign-in page. I type my password and the current six-digit code. The attacker relays both immediately, while that code is still valid, and receives an authenticated session as me. The attacker does not break the TOTP algorithm. The attacker steals one live code together with the password. A FIDO2 security key would refuse to sign the login for the phishing site, so this bypass depends on the software TOTP factor.

4 Authentication in UBC

[15 pts] Question 8 Comment on the two answers from the UBC website, and quote the NIST recommendation that applies to each one.

"Why do I have to change my CWL password?"
My comment: UBC's FAQ says that, for security reasons, users are prompted to replace their CWL password once a year, and that changing it even more often would be safer. A fixed expiry does not add much security. People who must invent a new password on a schedule usually keep the old one and tweak it, for example by advancing a year or a trailing digit. Anyone who has already seen the previous password can try those small edits. A reset is worth requiring when there is evidence that the password has been compromised, not because a year has passed.
NIST recommendation (quote and section number): NIST SP 800-63B-4, Section 3.1.1.2: "Verifiers and CSPs SHALL NOT require subscribers to change passwords periodically. However, verifiers SHALL force a change if there is evidence that the authenticator has been compromised."

"What are the CWL password requirements?"
My comment: The passphrase option is the part that fits current guidance: 16 to 50 characters, with spaces allowed inside the phrase, so length can carry the strength. The short option works against that guidance. It accepts 8 to 15 characters only if the password contains an uppercase letter, a lowercase letter, a digit, and a symbol. That mixture rule pushes people toward predictable patterns such as a word with a capital and a digit stuck on the end. Eight characters also meets NIST's minimum only when the password is one factor inside MFA. On its own, NIST requires at least 15 characters, so the short option falls short of the single-factor rule. The passphrase ceiling of 50 characters is below NIST's recommended allowance as well. That allowance is a SHOULD, not a SHALL, so the cap misses a recommendation rather than breaking a mandatory limit.
NIST recommendation (quote and section number): NIST SP 800-63B-4, Section 3.1.1.2: "Verifiers and CSPs SHALL require passwords that are used as a single-factor authentication mechanism to be a minimum of 15 characters in length." "Verifiers and CSPs SHALL NOT impose other composition rules (e.g., requiring mixtures of different character types) for passwords." "Verifiers and CSPs SHOULD permit a maximum password length of at least 64 characters."

Generative AI Disclosure

I told the assistant the logic, and the assistant implemented the code. For Question 9, I told it to build the super_admin message and its SHA-256 first, put that suffix in the password

so the server CBC-encrypts it, then keep only those ciphertext blocks and drop the rest. The assistant wrote that code. The code is in `https://github.com/LyuLucas1207/Authentication-And-Hashing`. I ran the tests and decided the written answers in this PDF.